

Distributed Systems

Indice

Intro	3
Modeling	3
Software architecture	3
Run-time architecture	4
Interaction model	5
Failure model	6
Communication	6
RPC	6
Binding	7
Dynamic activation	7
Lightweight RPC	7
Asynchronous RPC	7
RMI	8
Message oriented communication	8
MPI	8
Message queuing	8
Publish-subscribe	9
Stream-oriented communication	10
Naming	10
Flat naming	11
Structured naming	12
Attribute based naming	13
Removing unreferenced entities	13
Reference counting	13
Reference listing	13
Identifying unreachable entities	13
Fault tolerance	14
Protection against process failures	14
Agreement in process groups	14
Agreement with byzantine failures	15
Agreement in asynchronous systems	15
Reliable group communication	16
Non-faulty processes	16
Faulty processes	16
Recovery	17
Synchronization	18
How we measure time	18
Synchronizing physical clocks	18
GPS	19
Christian's algorithm	19

Berkeley	19
NTP	19
Logical time	20
Scalar clocks	20
Vector clocks	21
Mutual exclusion	21
Centralized	21
Scalar clocks	21
Token ring	22
Comparison	22
Leader election	22
Bully election	22
Ring-based	22
Capturing global state	22
Chandy-Lamport's algorithm	23
Termination detection	23
Distributed transactions	24
Atomicity	24
Controlling concurrency	24
Distributed deadlocks	25
Distributed agreement in practice	26
Commit protocols	26
Two-phase commit (2PC)	26
Three-phase commit (3PC)	26
CAP theorem	28
Replicated state machines	28
Paxos	28
Raft	28
Use in distributed DBMS	29
Byzantine conditions	29
Blockchains	29
Peer-to-peer	30
Centralized search (Napster)	30
Query flooding (Gnutella)	30
Hierarchical flooding (Kazaa)	31
Collaborative systems (BitTorrent)	31
Secure storage (Freenet)	32
Distributed hash tables (Chord)	32
Replication	33
Models	33
Single leader protocols	33
Multi-leader protocols	34
Leader-less protocols	34
Data-centric models	34
Eventual consistency	35
Client-centric models	35
Design strategies	36
Case studies	36
Distributed databases	36
Spanner	37
Calvin	37
VoltDB	37
Big data platforms	37

MapReduce	38
Scheduling	38
Fault-tolerance	38
Stragglers	38
Conclusions	39
Beyond MapReduce	39
Apache Sparck	39
Apache Flink	39
Comparisons	39

Intro

Definition of distributed system:

1. A collection of independent computers that appears to its users as a single coherent system
2. One in which hardware or software components located at networked computers communicate and coordinate their actions only by passing messages
3. One on which I cannot get any work done because some machine I have never heard of has crashed

Some defining features are:

- Concurrency
- Absence of a global lock
- Independent (and partial) failures

And some common problems for DSs are:

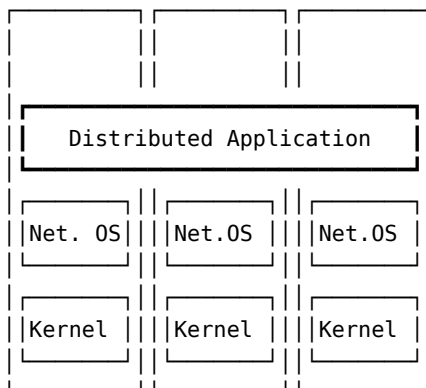
1. Heterogeneity: different machines have different specs/os/architecture with components written in different programming languages
2. Openness: whether a system can be extended and re-implemented in various ways
3. Security: needs to provide CIA and be resilient to attacks
4. Scalability: ability to increase the size of the system in terms of users/resources, geographic span, administrative span
5. Failure handling: detect/mask/tolerate/recover from failures
6. Concurrency
7. Transparency: we need to hide to the upper layers some details of the distributed system (by providing abstractions) as much as possible

Modeling

Software architecture

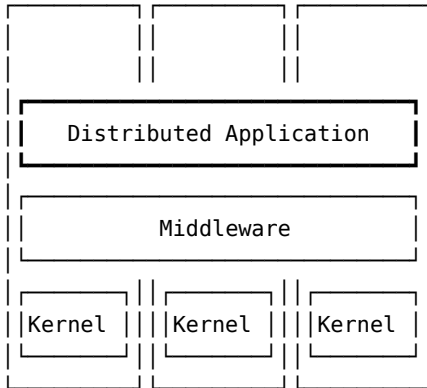
We have **two usual architectures**:

1. **Network OS based:**



2. **Middleware based:** Middleware provides “business-unaware” services through a standard API. Usually it provides:

- Communication and coordination services
- Special application services
- Management services



Run-time architecture

The runtime-architecture is usually one of a small set of very well-known architectural styles like:

- **Client-server**
- **Service Oriented:** built around the concepts of
 - **Services:** loosely coupled units of functionality
 - **Service providers:** export services
 - **Service brokers:** hold the description of available services
 - **Service consumer:** those who bind and invoke the services they need

Orchestration is the process of invoking a set of services in an ad-hoc workflow to satisfy a given goal

A **web service** is “a software designed to support interoperable machine-to-machine interaction over a network”. Its **interface** is described in **WSDL** and is invoked through the SOAP protocol. **UDDI** describes the **rules that allows web services to be exported and searched through a registry**.

- **REST:** *REpresentational State Transfer* is a set of principles that define how web standards are supposed to be used
 - **Key goals:**
 - * **Scalability** of component interactions
 - * **Generality of interfaces**
 - * **Independent deployment of components**
 - * Intermediary components to **reduce latency**, **enforce security** and **encapsulate** legacy systems
 - **Main constraints:**
 - * **Client-server** and **stateless interactions** (state must be transferred from clients to servers)
 - * The **data** within a response must be **labeled as cacheable or not**
 - * Each component cannot “see” beyond the immediate layer with which they are interacting (**layered**)
 - * (Optional) Clients must support **code on-demand**
 - * Components expose a **uniform interface** with the following **constraints**:
 - **Identification of resources** (each resource must have an ID)
 - **Manipulation** of resources **through representations**
 - **Self-descriptive messages**
 - **Hypermedia as the engine of application state**
- **Peer-to-peer:** all components play the **same role**
 - **Why:**
 - * **Client-server does not scale well** due to the centralization (it was dominant when there were less clients)
 - * A server is a central point of failure

- * P2P leverages the availability of broadband and processing power at the end-host to overcome the scalability issues
- P2P **promotes the sharing of resources and services** through direct exchange between peers
- **Object-oriented:** the components **encapsulate a data structure providing an API to access and modify it**; each component is responsible for the integrity of said structure and hides the details of its implementation
 - Components interact through **RPC**
 - It is a “P2P” model but is often used to implement client-server

- **Data-centered:** components **communicate synchronously through a common (usually passive) repository through (usually) RPC**

Linda is a data sharing model proposed in the 80s, mostly used for parallel computation and recently revitalized in the context of distributed computing.

In Linda data is **contained in typed tuples, stored in a persistent global shared space** (tuple space). Standard operations are:

- `out(t)`: write the tuple in the tuple space
- `rd(p)`: returns a copy of a tuple matching the pattern, blocking if no match is found
- `in(p)`: like `rd(p)`, but withdraws the matching tuple from the space
- `eval(a)`: (optional) inserts a tuple generated by execution of the process `a`

Many variants are **also async and provide other non-standard primitives and non-trivial distributed applications**

- **Event-based:** components collaborate by **exchanging information about occurring events**
 - Components **publish notifications** about events they observe
 - They **subscribe** to the events they are interest about

The communication is:

- Purely message based
- Async
- Multicast
- Implicit
- Anonymous

It **requires a central event-broker** that keeps track of all the subscribe/publish relations. It is usually provided by middleware.

- **Mobile code:** it is based on the ability of **relocating code at runtime**. It can implement different paradigms based on how it is implemented:
 - **Client-server:** I tell the remote what I want to do and it does it (REST-like)
 - **Remote evaluation:** I send the code to the remote that executes it in its environment (e.g. Google Colab)
 - **Code on-demand:** I ask a remote to send me the code to execute (JavaScript on the web)
 - **Mobile agent:** I move the whole process (code + state) to a remote (sort of like VM migration) (used rarely)

A system provides:

- **Strong mobility:** if it has the ability to migrate both the code and the execution state to a different environment (like the mobile agent case);
- **Weak mobility:** if it has the ability to allow code movement across different environments (like code on-demand and remote evaluation).

Pros: the **mobility**; **Cons:** the **security**

Interaction model

Traditional programs can be described in terms of the algorithm they implement, however distributed systems are composed of many processes, which interact in complex ways. The behaviour of a distributed system is described by a distributed algorithm:

A distributed algorithm is a definition of the steps taken by each process, including the transmission of messages between them.

To formally **analyze the behavior of a distributed system we must distinguish** (at least in principle) between:

1. **Synchronous** distributed systems:
 - **The time to execute each step of a process has known lower and upper bounds**
 - **Each message transmitted** over a channel is received within a **known bounded time**
 - **Each process has a local clock whose drift rate** from real time has a **known bound**
2. **Asynchronous** distributed systems:
 - There are **no bounds** for process execution speeds, message transmission delays, clock drift rates

Any solution that is valid for an asynchronous distributed system is also valid for a synchronous one (but the vice versa is clearly false)

Failure model

Both processes and communication channels may fail. The **failure model defines the ways in which failure may occur**. We distinguish between:

1. **Omission failures**:
 - Processes: fail stop
 - Channels: send omission, channel omission, receive omission
2. **Byzantine failures**:
 - Processes: may omit intended processing steps or add more
 - Channels: message content may be corrupted, non-existent messages may be delivered or real messages may be delivered more than once
3. **Timing failures** (only for synchronous systems):
 - occur when one of the time limits defined for the system is violated

Distributed consensus has been formally demonstrated to be impossible. To make the problem possible we need to either:

1. Change the assumptions (e.g. make links reliable)
2. Reduce the guarantees (e.g. only probabilistic instead of deterministic)

We can describe algorithms only for synchronous systems, with bounds large enough that they work even in the asynchronous world with high enough probability.

Communication

Middleware includes common services and protocols that can be used by many different applications. We can see the **middleware layer as a protocol layer**.

Middleware may **offer different form of communication** like:

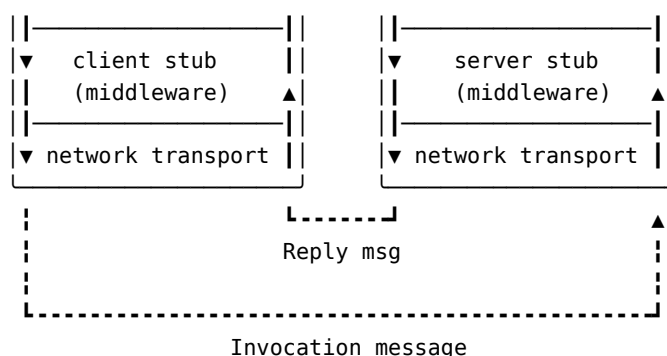
- **Transient/persistent**: transient is communication that requires both parties to be available at the same time (e.g. a phonecall)
- **Synchronous/asynchronous**: synchronicity w.r.t the middleware software, reception or response

We will see some communication methods.

RPC

In a local procedure call, parameters are passed on the stack, in different ways (value, reference, copy/restore). **What if the procedure we executed was on a remote machine?** The **remote communication can be hidden by the procedure call mechanism**.





In RPC **parameter passing** poses **two problems**:

1. **Structured data must be flattened** in a byte stream (**serialization**)
2. **Hosts may use different data representation** and proper conversions are needed (**marshalling**)

Middleware provides **automated support** by automatically generating marshalling/serialization code **from the function signature**.

RPC is enabled by a **platform and language independent representation of the procedure's signature**, a Interface Definition Language (**IDL**), and a **data representation format** to be used during communication. It **separates the service interface from its implementation**. The IDL comes with mappings onto target programming languages.

We still have the **problem of passing parameters by reference/pointer**. Often this is **unsupported**. A possibility is to change the semantics and make the passing by value/result.

The gold standard is Sun Microsystem's RPC (also called Open Network Computing RPC). The Distributed Computing Environment is another set of standards, more modern than sun's.

Binding

RPC poses the problem of **finding which server provides a given method** (binding the client to the server). This problem is divided in **two other sub-problems**:

1. Find out **where the server process is**
2. Find out **how to establish communication** with it

Sun's solution is to include a daemon (portmap) that is basically a forwarding table: it **maps procedures to server/port couples**. This solution, however, **solves only the second problem** because the client needs to know in advance where the service resides.

DCE's solution is a **directory service**, aka a binder daemon, that enables location transparency. Client need to know only the location of the directory service. It can then query the directory to get the service's location. Servers simply register their presence in the directory.

Dynamic activation

To start the server process on demand, we can use a standard daemon (inetd) that **launches the needed process only when a packet arrives on the configured port**.

Lightweight RPC

RPC can be used in any case where there is a need for communication between processes. If the two processes are **running on the same machine**, we can build a **lightweight variant** of the RPC protocol using e.g. **shared memory maps** for communication.

Asynchronous RPC

RPC is by definition **synchronous** since it is trying to emulate the natural call behaviour. We can also **make RPC async by modifying the call semantics** (introducing callbacks on procedure termination etc).

We can also implement **batched/queued RPC** to achieve very efficient communication and high performance.

RMI

It starts from the **same idea as RPC**, but it uses different programming constructs. The **aim is to obtain the advantages of OOP in the distributed setting**.

An important difference w.r.t RPC is that **remote object references can be passed around: we need to maintain the aliasing relationship**. This is easier in a OOP setting since **we interact with objects only through their interfaces**. This means that **passing a reference to an object is equivalent to passing a reference to a proxy implementing the remote call procedure**. In contrast to RPC, **in RMI true pass-by-value is instead impossible**.

In RMI, we can use the fact that the separation between interface and implementation is a basic principle and have **very rich IDLs that can model complex relationships and behaviours** (e.g. inheritance, exceptions).

Implementation is similar to that of RPC, so much so that often it is implemented on top of a RPC layer.

Two popular RMI middlewares are:

1. Java RMI (everything must be java)
2. OMG Corba (multilanguage/multiplatform)

Message oriented communication

RMI/RPC foster a synchronous model: it is natural but works only in a point-to-point synchronous mode that couples tightly caller and callee limiting to rigid architectures.

Message oriented architecture are:

- Centered around the (simpler) notion of **one-way message/event**
- Usually **async**
- Often **supporting persistent communication and multi-point communication**
- Brings **more decoupling**

MPI

The **most straightforward way** of doing message oriented communication is **message passing** (see UDP et al). They are typically mapped on/provided by the **OS** (with e.g. **sockets**) **or by middleware** providing an interface (**MPI**).

Message queueing and publish/subscribe are provided at the **middleware layer** by several communication servers, nowadays called an **overlay network**, which form a **network on top of the network**.

Since **sockets are lowlevel and protocol independent**, they can be hard to use. In high performance networks, **we may need higher level primitives for specific types of communication** providing different services besides pure r/w. **MPI** is the **platform independent answer**.

Communication takes place within a known group of processes. Each process within a group is assigned a local ID:

1. The **pair** (groupID, procID) represents a **source/destination address**
2. Messages can be also sent in **broadcast**

There is **no** support for **fault tolerance**.

Message queueing

It is a mechanism to enable **asynchronous point-to-point communication**. Typically it **only guarantees eventual insertion into the recipient queue**, not its behaviour. Communication is **decoupled in time and space**. Each component holds an **input queue and an output queue, which are named**.

We can look at the architecture more in depth:

1. **Queues are identified by symbolic names**
 - We have the **need for a lookup service** that converts queue addresses into network addresses
2. Queues are **manipulated by queue managers** (acting like relays)
3. Relays are **organized in an overlay network**

- Messages are **routed by using application level criteria and by relying on partial knowledge of the network**. This **improves fault tolerance** and provides applications with **multi-point** without relying on IP multicast at the price of coupling.
4. **Message brokers provide application level gateway supporting message conversion**

Publish-subscribe

Application components can **publish asynchronous event notification and/or declare their interest in event classes by issuing a subscription**. Subscriptions are collected by an **event dispatcher component** (centralized or distributed), responsible for routing events to all matching subscribers.

Communication is **transiently asynchronous, implicit and multipoint**. It also offers a **high degree of decoupling among components**.

To **describe a subscription**, we use a **subscription language**. The expressiveness of it allows one to distinguish between **two types of subscriptions**:

1. **Subject/topic based**: the **set of subjects is determined a-priori** (analogous to multicast).
2. **Content based**: subscriptions contain **expressions** (event filters) that allow clients to **filter events based on the content**

These two types of subscriptions can be **combined**. The **trade-off** between the two is mainly in complexity vs. expressiveness.

In event-based systems a special component of the architecture, the **event dispatcher**, is **in charge of collecting subscriptions and routing event notifications based on such subscriptions**. Its implementation can be:

1. **Centralized**: a **single component** is in charge of collecting subscriptions and forward messages to subscribers
2. **Distributed**: a **set of message brokers organized in an overlay network** cooperate to collect subscription and route messages

In case of a distributed event dispatcher, the **topology of the overlay network can be either acyclic or cyclic**. This influences how we deliver messages:

1. Acyclic:

- **Message forwarding**: every broker stores only subscriptions coming from directly connected clients, message are forwarded broker to broker and delivered to clients only if there are subscribed
- **Subscription forwarding**: complementary to message forwarding, every broker forwards subscriptions to the others (never twice on the same link); each time a broker receives a message it must match it against the list of received filter to determine the list of recipients
- **Hierarchical forwarding**: we elect one broker as the root of the acyclic graph (we can now treat it as an overlay tree); both messages and subscriptions are forwarded by brokers towards the root of the tree, messages flow downwards only if a matching subscription had been received along that route

2. Cyclic:

- **Distributed hash table based approach**: a DHT organizes nodes in a structured overlay allowing efficient routing toward the node having the smaller ID greater or equal than any given ID. We can easily build a subject based system using this.

To subscribe for messages having a given subject S :

- Calculate a hash of the subject Hs
- Use the DHT to route toward the node $succ(Hs)$
- While flowing toward $succ(Hs)$ leave routing information to return messages back

To publish messages having a given subject S :

- Calculate a hash of the subject Hs
- Use the DHT to route toward the node $succ(Hs)$
- While flowing toward $succ(Hs)$ follow back routes toward subscribers

- **Content based routing**: we try to emulate a network by using routing and forwarding strategies.

Forwarding strategies:

- Per-source forwarding: every source defines a shortest path tree, forwarding tables keep information (next-hop and predicate) organized per source
- Per-receiver forwarding: the source of the messages calculates the set of receivers and adds them to the header of the message (thus every node know the subscription state of all the network); at each hop the set of recipients is partitioned in two tables:
 - * The routing table
 - * A forwarding table with the predicate for each network

Routing strategies: distance vector and link-state

Complex event processing CEP systems adds the **ability to deploy rules that describe how composite events can be generated from primitive (or composite) ones**. Open issues:

1. The **rule language**: need for balance between expressiveness and processing complexity
2. **Processing engine**: how to efficiently match incoming (primitive) events to build complex ones
3. **How to distribute processing**

Stream-oriented communication

The main concept is the **data-stream**, which is a **sequence of data units**. **Time usually does not impact the correctness of the communication, just the performance** (but it is **not always** the case). We have various **transmission modes**:

1. **Asynchronous**: the data items in a stream are transmitted one after the other without timing constraints
2. **Synchronous**: there is a maximum end-to-end delay for each unit
3. **Isochronous**: there are both a maximum and minimum end-to-end delay for each unit

The stream can be **simple or complex** (i.e. composed of different substreams).

Non-functional requirements are often expressed as **Quality of Service (QoS)** requirements:

- Required **bit rate**
- Maximum **delay to setup** the session
- Maximum **end-to-end delay**
- Maximum variance in delay (**jitter**)

IP is a best-effort protocol. It supports a Differentiated Services field in its header (TOS bits), but it is not necessarily supported by routers. This means that we need to **enforce QOS at the application level**. We have several **techniques**:

1. **Buffering**: control max jitter by sacrificing session setup time
2. **Forward error correction**: once we recognize that an error occurred (e.g. packet drops), we try to correct it and bring the application in a correct state

This is the opposite of backward error correction, which is basically what TCP does on packet drops.

3. **Interleaving data** to mitigate the impact of lost packets

Synchronizing two or more streams is **challenging**:

1. Depending on the type of stream it can mean different things
2. Synchronization may take place at the sender or the receiver side
3. It may happen either in middleware or in application code.

Naming

Names are used to **refer to entities**, which are usually **accessed through an access point**, which is itself a **special entity characterized by an address**. An address is a special case of name. The same **entity can be accessed through several access points** at the same time and it **can change its access points during its lifetime**. Thus, it is not convenient to use the address of its access point as a name for an entity but it is better to use **location-independent names**.

We can distinguish between:

1. Global and local names

- Global denotes the same entity everywhere, local names only in the context where it is used

2. Human-friendly vs machine-friendly

Resolving a name directly into an address does not work with mobility. Identifiers are names such that:

- They **never change**
- **Each entity** has exactly **one identifier**
- An identifier is **never assigned to another entity**

Using identifiers enables to **split the problem of mapping a name to an entity and the problem of locating the entity**.

Name resolution is the process of **obtaining the address of a valid access point of an entity having its name**. The way we do it depends on the nature of the **naming schema** employed:

1. **Flat naming**
2. **Structured naming**
3. **Attribute-based naming**

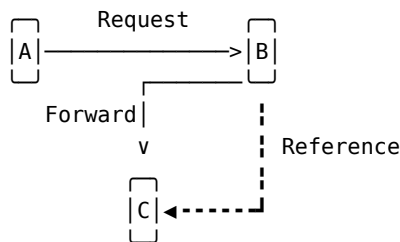
Flat naming

In a flat scheme **names are simple strings with neither structure nor content**. The name resolution process can be:

1. Simple: designed for small-scale environments

- **Broadcast based**: similar to ARP, send find messages in broadcast and interested hosts reply
- **Multi-cast based**: same as broadcast, but use multi-cast to reduce the scope of the search
- **Forwarding pointer based** (for mobile nodes): leave reference to the next location at the previous location; example:

A wants to contact C which was previously at address B.



This method can lead to long chains a high latencies due to dereferencing

2. Home based: rely on one home node (assumed stable) that knows the location of the mobile unit

This extra call to the home increases latency and has bad scalability (two entities near by still need to call the home which may be far away). Moreover the address of the home is fixed and must live for the whole lifespan of the entity.

3. Distributed hash table: it is a very nice way to store names since it is a key-value store (the key is the name, value is the address)

An implementation can be with different topologies organized in an overlay network

4. Hierarchical approach: the network is divided into a hierarchy of domains with the root domain spanning the entire network. Each domain has an associated directory that keeps track of the entities in that domain.

The root has entries for every entity, entries point to the next sub-domain. A leaf domain contains the address of an entity (entities may have different names in different leaf domains).

Lookup may start anywhere. We first look locally; if we do not find anything we forward the lookup to the parent; if we find something we forward to the child leaf holding the concrete entry.

With this method we achieve locality, however higher levels of the tree have more and more information, with the root having information on all entries.

Updates to the hierarchy are done through insert request from the new location. Records can be created bottom up or top down. Deletion proceeds from the old node up until a node with multiple children is reached.

Caching can reduce lookup times. Also it is possible to shortcut search if information about mobility patterns are available. The root node can be distribute to avoid bottlenecks.

Structured naming

Names are **organized in a name space**, a **labeled graph composed of leaf and directory nodes**. A **leaf** represents a **named entity**. A **directory** has a **number of labeled outgoing edges**, each pointing to a different node. **Resources are referred to through path names** (absolute or relative). **Multiple path names may refer to the same entity** (hard linking) or **leaf nodes may store absolute path names of the entity they refer to instead of their identifier/address** (symbolic linking).

Name spaces for a large scale, possibly worldwide, distributed system are often **distributed among different name servers, usually organized hierarchically**. Name spaces are made of **layers**:

1. **Global level**: high-level directory nodes; these directory nodes have to be jointly managed by different administrations
2. **Administration level**: mid-level directory nodes that can be grouped in such a way that each group can be assigned to a separate administration
3. **Managerial level**: low-level directory nodes within a single administration; main issue is effectively mapping directory nodes to local name servers

The **higher part of the graph is very stable** so we can use caching to improve performance. The **lower parts are more volatile** so its better to achieve faster lookups than to use caching.

The best known example of structured naming is **DNS**. Lets have a look at what DNS does:

Item	Global	Administration	Managerial
Scale	Worldwide	Organization	Department
N. of nodes	Few	Many	Vast
Responsiveness to lookups	Seconds	Milliseconds	Immediate
Update propagation	Lazy	Immediate	Immediate
N. of replicas	Many	None or few	None
Client-side caching used?	Yes	Yes	Sometimes

We have two way of resolving DNS names:

1. Iterative: we iteratively go down the hierarchy until we find an authoritative server
2. Recursive: The root server recursively asks child directories to find the authoritative server and then responds to the query

Recursive resolution has some advantages: communication costs may be reduced and caching is more effective. However it puts higher load on servers

DNS has very poor performance if we do not use neither caching or replication. However it uses those, so its fast. The root servers uses IP anycast to route queries among the various replicas.

DNS works well on the assumptions that:

1. The lower levels are stable
2. Content of the managerial layer changes often, but requests are server by name servers in the same zone, therefore updates are efficient

If we allow a host to move, we do not have major problems if it remains in the same domain. If it moves to another domain we get several problems affecting locality and the speed of lookups.

Attribute based naming

Problem: as more information is made available, it becomes **important to effectively search for items**.

Solution: **refer to entities** not with their name but with a **set of attributes**, which encode their properties.

Attribute based naming systems are **usually called directory services** and usually **implemented by using DBMS technology**.

Best know example of this naming is **LDAP**. Each LDAP directory consist of a number of records and is made of <attribute, value> pairs. Each attribute has a type and can be single-values or multiple-valued. The collection of all records in a LDAP directory servisse is called Directory Information Base (DIB). Each record has a unique name defined as a sequence of naming attributes (aka relative distinguished name). This allows us to build a directory information tree, meaning a node in a LDAP naming graph can simultaneously represent a directory in a traditional sense (in a hierarchical name space).

Removing unreferenced entities

Entities accessed through stale bindings should be removed. Automatic garbage collection is common in conventional systems. **Distribution greatly complicates matters**, due to lack of global knowledge about who's using what, and to network failures.

Reference counting

The object (e.g., in its skeleton) **keeps track of how many other objects have been given references**. Reliability (**exactly-once message delivery**) must be ensured. **Race condition** when passing references among processes are possible (not a problem in non-distributed systems).

Weighted reference counting tries to circumvent the race condition by communicating only counter decrements. It requires an additional counter. Removing a reference subtracts the proxy partial counter from the total counter of the skeleton: when the total and partial weights become equal, the object can be removed. The problem of this method is that **only a fixed number of references can be created**. To circumvent it we would need to create a chain of indirection.

Reference listing

Instead of keeping track of the number of references, **keep track of the identities of the proxies**. Advantages:

1. **Insertion/deletion of a proxy is idempotent**
 - Insertion and deletion of references must still be acknowledged, but requests can be issued multiple times with the same effect
2. Easier to **maintain the list consistent w.r.t. network failures** by e.g. periodically pinging clients

Still suffers from **race conditions** when copying references.

Identifying unreachable entities

To find unreachable entities we have **two main approaches**:

1. **Tracing-based** garbage collection techniques: require knowledge about all entities, therefore they have inherent **poor scalability**
2. **Mark-and-sweep**:
 - On centralized systems:
 1. First phase marks accessible entities by following references
 - Initially nodes are white
 - A node is colored gray when reachable from a root
 - A node is colored black after it turned gray and all its outgoing references have been marked gray
 2. Second phase exhaustively examines memory to locate entities not marked and removes them
 - On distributed systems: garbage collectors run on each site, in theory it can work however we need to freeze the distributed system and maintain the reachability graph stable.

In practice it is never done.

Fault tolerance

To have a dependable system we need: **availability, reliability, safety, maintainability**. Note that availability and reliability are different: if a system goes down for one millisecond every hour, it has an availability of 99.999%, but is still highly unreliable.

A system fails when it is not able provide its services. **A failure is the result of an error. An error is caused by a fault.** Some faults can be avoided, others cannot. Building a **dependable system demands the ability to deal with the presence of faults**. A system is said to be **fault tolerant if it can provide its services even in the presence of faults**.

Faults can be classified according to the frequency at which they occur:

1. **Transient** faults occur once and disappear
2. **Intermittent** faults appear and vanish with no apparent reason
3. **Permanent** faults continue to exist until the failed components are repaired

The general techniques for building fault tolerant systems are:

1. **Information redundancy**
2. **Time redundancy**
3. **Physical redundancy**

Protection against process failures

In a client-server system, for example, **either the server or the client can crash**. Clients **cannot distinguish whether a server is unreachable, has crashed before or after executing the functions, or the reply has been lost**. We can **only ensure at-most-once or at-least-once execution**, no scheme can reliably ensure exactly-once computation. A **computation started by a dead client is called an orphan**. Orphans still consume resources on the server, so **they must be removed**:

1. **Extermination**: requests are logged by the client and orpahns killed after a reboot
2. **Reincarnation**: when a client reboots, it starts a new epoch and sends a broadcast messages to servers, who kill old computations started on his behalf
3. **Gentle reincarnation**: as normal reincarnation, but servers kill old computations only if the owner cannot be located
4. **Expiration**: remote computations expire after some time; clients wait to reboot to let remote computations to expire

Reincarnation can solve most problems, however a lot of bookkeeping is required and not all issues can be solved.

Redundancy can be used to mask the presence of faulty processes, with **redundant process groups**: the work that should be done by a process is taken care by a group of processes; the healthy processes can continue to work when some other fails. **We have two possible group hierarchies: flat or hierarchical** (coordinator with workers).

To use groups, we must **keep track of which processes constitute each group**. Having a **coordinator seems the best choice**, however the **coordinator becomes the single point of failure** of the whole system. **Distributed membership management in flat groups requires that join and leave announcements be reliably multicast**.

How large should a group be?

- If processes fail silently, then $k + 1$ **processes are required to achieve k -fault tolerance**
- If **failures are byzantine**, it becomes **worse**: $2k + 1$ **processes are required to achieve k -fault tolerance** (to achieve a working voting system)

In practice, **we cannot be sure that no more than k -processes will ever fail simultaneously**.

Agreement in process groups

A number of tasks may require that the members of a group agree on some decision before continuing. Because we are dealing with faults, we want all non-faulty processes to reach an agreement.

This is the **consensus problem**; the non-faulty processes must agree on a valid value (considered valid by a validity condition). More precisely we have the **following conditions**:

1. Each **process starts with some initial value**
2. All **non-faulty processes have to reach a decision based on these values**
3. The following must hold:
 - **Agreement**: no two processes decide on different values
 - **Validity**: if all processes start with the same value v , then v is the only possible decision value
 - **Termination**: all non-faulty processes eventually decide

Consensus is **not possible in the presence of arbitrary failures**. We will consider that **communication is reliable, but processes are allowed to fail**. Our assumption then become:

1. We **consider a synchronous system** in which all processes evolve in synchronous rounds
 - A message sent by a process is received within the same round by the recipient
 - Processes may fail at any point, by stopping taking steps
2. We want our processes to reach an agreement according to the definition of consensus

This problem is simpler: it **can be solved provided that the processes take at least $f + 1$ rounds** with f a bound on the number of failures. It is also only **sufficient that a single process is non faulty**.

A simple algorithm to solve this problem is the **FloodSet algorithm**.

- Let v_0 be an pre-specified default value. Each process maintains a variable $W \subset V$ initialized with its start value. The following is repeated for $f + 1$ rounds
 1. Each process sends W to all other processes
 2. It adds the received sets to W
- The decision is made after $f + 1$ rounds:
 - If $|W| = 1$: decide on W 's element
 - If $|W| > 1$: decide on v_0 (or use the same function to decide, e.g. max)

Lemma: FloodSet correctness If no process fails during a particular round $r : 1 \leq r \leq f + 1$, then $W_i(r) = W_j(r)$ for all i and j that are active after r rounds.

Proof Suppose that $W_i(r) = W_j(r)$ for all i and j that are active after r rounds. Then for any round $r_1 : r \leq r_1 \leq f + 1$, the same holds, that is, $W_i(r_1) = W_j(r_1)$ for all i and j that are active after r_1 rounds. If processes i and j are both active after $f + 1$ rounds, then $W_i = W_j$ at the end of round $f + 1$.

We can improve the complexity of FloodSet by optimizing it a bit: we broadcast W at the first round and we broadcast new values only when we receive new values.

Agreement with byzantine failures

Our algorithm **does not consider byzantine failures**. Our conditions now become:

- Agreement: no two non-faulty processes decide on different values
- Validity: if all non-faulty processes start with the same value v , then v is the only possible decision value
- Termination: all non-faulty processes eventually decide

Lamport's algorithm is a solution. It is **formulated with generals that need to decide total number of troops based on troops strength**; however **we have some traitors** that will misreport. The **steps** for a 4 generals and 1 traitor are:

1. Send troop strength to others
2. Form a vector with the received values
3. Send a vector to others
4. Compute the vector using majority for each vector position

Lamport (1982) showed that **if there are m traitors, $2m + 1$ loyal generals are needed for an agreement to be reached**, for a total of $3m + 1$. There is **no way for 1 and 2** to determine a vector which is both correct and equal to that computed by the others.

Agreement in asynchronous systems

Let us consider the problem of reaching an agreement between n processes with 1 faulty process in an asynchronous system.

Fischer, Lynch, and Paterson proved that no solution exists. The result was proved in the case of crash failures but it also extends to byzantine failures since a byzantine program can simulate a crashed one, implying that if a solution existed for byzantine faults it would also exist for crash faults.

Reliable group communication

It is of critical importance if we want to exploit process resilience by replication. **Achieving reliable multicast through multiple reliable point-to-point channels may not be efficient or enough.**

We assume **two cases**:

1. **Fixed groups, non-faulty processes**
2. **Faulty processes, groups are allowed to change**

Non-faulty processes

It is **easy to implement on top of unreliable multi-cast** with either:

- **Positive acknowledgment**: doesn't scale well since we could DoS ourselves with ACKs
- **Negative acknowledgment**:
 - **Non-hierarchical feedback control**: assume messages are delivered correctly and broadcast a NACK after a random timeout in case something goes wrong. The NACK broadcast is also used by other senders to suppress their feedback in case they also encountered some problem
 - **Hierarchical feedback control**: receivers are recursively organized in subgroups headed by coordinators. The coordinator can adopt any strategy in the group and can request retransmissions to parent coordinators. The problem is that the hierarchy has to be established and maintained.

Faulty processes

To address this we **need that a message is delivered either to all the members of a group or to none and that the order of messages is the same at all receivers**. This requirement is called **atomic multicast**.

Ideally we would like that any two processes that receive the same multicast messages or observe the same group changes to see the corresponding events in the same order and that a **multicast be delivered to the full membership**. This is called **close synchrony** and **cannot be achieved** in the presence of failures.

A mechanism for detecting failures is needed. However, even if we can detect failures correctly we cannot know whether a failed process has received and processed a message.

We can **weaken our model** to:

1. **Crashed processes are purged** from the group and have to join again
2. **Messages from a correct process are processed by all correct processes**
3. **Messages from a failing process are processed either by all correct member or by none**
4. **Messages are received in a specific order**

To our multicast primitive it is useful to adopt a model which distinguishes between receiving and delivering a message: **messages received by the communication layer are buffered there and later delivered, when some condition holds**.

A **group view** is the **set of processes to which a message should be delivered as seen by the sender at sending time**. The minimal ordering requirement is that **group view changes be delivered in a consistent order with respect to other multicasts and with respect to each other**.

This, together with the previous requirements, leads to a form of reliable multicast which is said to be **virtually synchronous**.

We say that a **view change occurs when a process joins or leaves the group**, possibly crashing. **All multicast must take place between view changes**, which themselves can be seen as another form of multicast messages. We must guarantee that **messages are always delivered before or after a view change**. If the **view change is the result of the sender** of a message *m* leaving, the message is either delivered to all group members before the view change is announced or it is dropped.

Retaining the virtual synchrony property, we can identify **different orderings** for the multicast messages:

- **Unordered**
- **FIFO**
- **Causally ordered**

In addition, the above orderings can be combined with a **total ordering requirement**: whatever ordering is chosen, **messages must be delivered to every group member in the same order**. **Atomic multicast** is defined as a **virtually synchronous reliable multicast offering totally-ordered delivery of messages**.

One implementation of virtual synchronicity is ISIS. In practice, although each transmission is guaranteed to succeed, there are no guarantees that all group members receive it; the sender may fail before completing its job. ISIS must **make sure that messages sent to a group view are all delivered before the view changes**:

1. **Every process keeps a message m until it is sure that all the others have received it**. Once this happens, m is said to be **stable**
2. We assume that **processes are notified when messages become stable** and that **they keep them in a buffer until that time**

To ensure that multi-casts are between view changes, we follow this generic scheme:

1. We assume that processes are notified of view changes by some (possibly distributed) component
2. When a process receives a view change message, it stops sending new messages until the new view is installed and it multicasts all pending unstable messages to the non faulty members of the old view, marks them as stable and multicasts a flush message
3. Eventually all the non faulty members of the old view will receive the view change and do the same
4. Each process installs the new view as soon as it has received a flush message from each other process in the new view. Now it can restart sending new messages

If we **assume** that the **group membership does not change during the execution of the above protocol** we have that at the end all non faulty members of the old view receive the same set of messages before installing the new view.

Recovery

When processes resume working after a failure, they have to be **taken back to a correct state**. We have two methods:

1. **Backward recovery**: The system is brought back to a **previously saved correct state**
2. **Forward recovery**: The system is brought into a **new correct state** from which execution can be resumed

Recovering a previous state is only possible if that state can be retrieved. We have two methods of retrieving a previous state:

1. **Checkpointing** consists in **periodically saving the distributed state** of the system to stable storage
 - It is **very expensive**
 - Each node of the system takes snapshots of its private state. The problem is finding the most recent **consistent cut** (i.e. a moment where the overall state of the system is consistent), which is a cut where all messages have been sent and delivered. This means that **we may need to roll back each system until we find a set of checkpoints that form a consistent cut (domino effect)**

This is **not trivial to implement** since we need:

1. **Tag intervals** between two checkpoints
2. **Each message exchanged** by the process must **record a reference** to the **interval**
3. Implement **dependency information between intervals** (constructed by each node with info embedded in messages)

When a **failure** occurs, the **recovering process broadcasts a dependency request to collect dependency information**. The **recovery line is computed** starting from **two graphs computed from the info received**: dependency graph and checkpoint graph.

- We can also do **coordinated checkpoints**: the **coordinator sends a checkpoint request** and on receive the nodes take a checkpoint, ACKing when done. This requires electing a coordinator.

2. With **logging**, events (messages) are recorded to stable storage so that they can be replayed when recovering from a failure. We can also **combine it with checkpointing** by starting from a checkpoint and replaying it from the log.

Logging works if the system is **piecewise deterministic**: execution proceeds deterministically between the receipt of two messages. Logging must be done carefully.

Each message's header contains all necessary information to replay it. We define a **message to be stable when it can no longer be lost**. For each unstable message we define **two sets of processes**:

- **$DEP(m)$** : processes that depend on the delivery of m , i.e., those which m has been delivered or to which m' dependent on m has been delivered
- **$COPY(m)$** : processes that have a copy of m , but not yet in stable storage. They are those processes that hand over a copy of m that could be useful to replay it

An **orphan process** is a process survived after one or more crashes in the system such that $\exists m : Q \in DEP(m)$ and all processes in $COPY(m)$ have crashed. This means that there is **no way for an orphan to replay the lost message**.

If each process in $COPY(m)$ has crashed then no surviving processes must be left in $DEP(m)$. This can be achieved by ensuring that $COPY(m) \supseteq DEP(m)$.

Based on this we can have **two schemes**:

1. **Pessimistic**: ensure that **any unstable message m is delivered to at most one process**
 - **Prevents orphans**, can also require logging the messages on the sender side
2. **Optimistic**: messages are **logged asynchronously, with the assumption that they will be logged before any faults occur**
 - **Can allow orphans**, we need to force them to roll back until they are not orphans anymore

Logging works very well in conjunction with coordinated checkpoints

Synchronization

Synchronization in distributed environments is **more difficult** due to the **absence of a clock, shared memory and possible partial failures**.

How we measure time

Time plays a fundamental role in many application. In distributed systems the **main problem is ensuring that all machines see the same global time**. Time is a tricky issue per se:

1. Up to 1940, time is measured astronomically (1 second is 1/86400 of the mean solar day)
2. Since 1948, time is measured physically using the oscillations of the Cesium-133 atom (International Atomic Time)

The skew between IAT and solar days is accounted for by UTC (Coordinated Universal Time) by introducing leap seconds.

Synchronizing physical clocks

First of all: computer clocks are not clocks, they are timers. A computer clock is defined by:

1. **Maximum clock drift rate ρ** (i.e. when the clock runs at a slightly different rate than that of the reference clock), which is constant
2. **Maximum allowed clock skew δ** (i.e. when the same signal arrives to different components at different times), is an engineering parameter that we decide during design

If **two clocks are drifting in opposite directions**, during a time interval Δt they **accumulate a skew of $2\rho\Delta t$** . This means that a **resynch is needed at least $\frac{\delta}{2\rho}$ seconds**.

We have **two ways of synchronizing** clocks:

1. **Against** a single external **reference** clock (accuracy)
2. **Among themselves** (agreement)

At least time monotonicity must be preserved.

GPS

One of the best way of synchronizing clocks is **using a side-effect of GPS**.

In GPS position is determined by triangulation from a set of satellites whose position is known. **Distance can be measured by the delay of the signal**, but the **satellites and receiver's clocks must be in sync**. Since they are not, we must take clock skew into account.

Let:

- Δ_r be the unknown deviation of the receiver's clock w.r.t. to the atomic reference on the satellites
- x_r, y_r, z_r the coordinates of the receiver
- T_i the timestamp of the message sent by a satellite i

Supposing that the messages sent by satellite i are received at T_r measured according to the receiver's time, which corresponds to T_{now} in real time, then:

1. $T_{now} = T_r - \Delta_r$,
2. $\Delta_i = T_r - T_i$ is the measured message delay
3. $c\Delta_i$ is the distance of the satellite

Putting everything together we have:

$$c\Delta_i = c(T_{now} - T_i + \Delta_r) = c(T_{now} - T_i) + c\Delta_r$$

Where the first addendum must be equal to the cartesian distance of the receiver from the satellite i .

Using **4 satellites**, we have **4 equations with four unknowns** (the xyz position and Δ_r) and we can solve them.

Christian's algorithm

One of the simplest algorithms for **synchronizing a clock against correct well-known source** (a time server).

Messages are assumed to **travel fast w.r.t. the required accuracy**. The **client measures** T_0 , the time at which sends a message to the time server, **and** T_1 , the time at which we receive the reply. We have the following relationships:

1. $T_1 = C_{UTC} + T_{round}/2$
2. $T_{round} = T_1 - T_0 - I$ with I the server processing time

Problems:

1. **Time might run backwards on client machine.** Therefore we need to introduce the adjustment gradually
2. It takes a **non-zero amount of time to get the message to the server and back**; solved by measuring the RTT and averaging over several measurements

Berkeley

Introduced in UNIX, we have an **active time server**: it **collects the time from all clients, averages it** and then **retransmits the required adjustment**.

NTP

Designed for UTC synchronization over large networks, runs on top of UDP. It is **organized hierarchically** in **strata**:

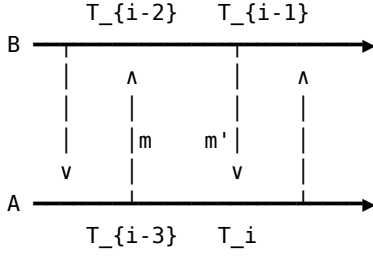
1. **Servers at the top of the hierarchy** (lower strata, with the lowest being stratum 1) are **directly connected to a UTC source**
2. The **higher the strata**, the **lower the accuracy** of the server
3. **Leaf servers execute on users' workstations**

We have **3 modes of operation**:

1. **Multicast (LAN)**: the server periodically multicasts their time to other computers on the network

2. **Procedure-call mode**: which is **similar to Christian's algorithm**
3. **Symmetric mode**: used when higher levels that need the highest accuracy

The procedure-call mode is a modified version of Christian's algorithm. **Servers exchange pairs of messages**, each **bearing timestamps** of recent events (the local time when the previous message between pairs was sent and received, and the local time when the current message was transmitted).



If t and t' are the messages' transmission times, and o is the time offset of the clock at B relative to A then:

$$T_{i-2} = T_{i-3} + t + o \quad T_i = T_{i-1} + t' - o$$

This leads to calculate the total transmission time d_i as:

$$d_i = t + t' = T_{i-2} - T_{i-3} + T_i - T_{i-1}$$

If we define $o_i = \frac{T_{i-2} - T_{i-3} + T_i - T_{i-1}}{2}$, from the first two formulas we have $o = o_i + \frac{t' - t}{2}$ and since $t, t' \geq 0$ we have:

$$o_i - \frac{d_i}{2} \leq o \leq o_i + \frac{d_i}{2}$$

Thus o_i is an estimate of the offset and d_i is a measure of the accuracy of this estimate.

Logical time

In many applications it is **sufficient to agree on a time, even if it is not accurate w.r.t. the absolute time**. What matters is often the **ordering and causality** relationships of events, rather than the timestamp itself. What we need is logical time.

Let us define the ***happens-before* relationship** as $e \rightarrow e'$ as follows:

1. If events e and e' occur in the same process and e occurs before e' , then $e \rightarrow e'$
2. If e is the sending of message and e' is the reception of a message, then $e \rightarrow e'$.

If **neither** $e \rightarrow e'$ nor $e' \rightarrow e$ then the two events are **concurrent** $e \parallel e'$.

***happens-before* is transitive and defines an ordering among events**, called **potential causal ordering**:

1. Two events can be related by the happens-before relationship even if there is no real (causal) connection among them
2. Also, since information can flow in ways other than message passing, two events may be causally related even neither of them happens-before the other

Scalar clocks

Lamport invented a simple mechanism, scalar clocks, by which the *happens-before* ordering can be captured numerically **using integers to represent the clock value**, with **no relationship with a physical clock** whatsoever. **Each process p_i keeps a logical scalar clock L_i that:**

1. L_i **starts at zero**
2. L_i is **incremented before p_i sends a message**
3. **Each message** sent by p_i is **timestamped** with L_i
4. **Upon receipt** of a message, p_i **sets L_i to $\max(\text{timestamp}, L_i) + 1$**

It can be easily shown, by induction on the length of any sequence of events relating two events e and e' , that $e \rightarrow e' \implies L(e) < L(e')$. This way **only partial ordering is achieved**; **total ordering** can be achieved by **attaching also process IDs** to the clocks.

Vector clocks

In **scalar clocks** we have that $e \rightarrow e' \implies L(e) < L(e')$, however the **reverse does not hold** (for example if $e \parallel e'$). A solution to this are vector clocks.

In vector clocks **each process maintains a vector** V_i of N (N is the number of processes) values such that:

1. $V_i[i]$ is the **number of events** that have occurred at p_i
2. If $V_i[j] = k$ then p_i **knows that k events have occurred at p_j**

Initially the **vectors start zeroed**. **Local events** at p_i causes an **increment** of $V_i[i]$. p_i **attaches the vector** V_i (post increment) as **timestamp** to messages; **when it receives** a message containing a timestamp t , it **sets** $\forall j \neq i \quad V_i[j] = \max(V_i[j], t[j])$ and **then increments** $V_i[i]$.

Vector clocks define a **partial ordering between timestamps**:

- $V = V' \iff \forall j \quad V[j] = V'[j]$
- $V \leq V' \iff \forall j \quad V[j] \leq V'[j]$
- $V < V' \iff V \leq V' \wedge V \neq V'$
- $V \parallel V' \iff \neg(V < V') \wedge \neg(V' < V)$

We can then **define an isomorphism** between the **set of partially ordered events** and **their vector clocks**:

1. $e \rightarrow e' \iff V(e) < V(e')$
2. $e \parallel e' \iff V(e) \parallel V(e')$

A slight **variation of vector clocks** can be used to **implement causal delivery** of messages in a totally distributed way: **increment clock only when sending a message; on receive, just merge, not increment**.

Mutual exclusion

Mutual exclusion is **required to prevent interference and ensure consistency of resource access**. On single systems is easier since we have a single clock and a shared memory, on distributed systems we have non of that. To guarantee mutual exclusion **we need to guarantee**:

1. **Safety**: at **most one process may execute** in the critical section at a time
2. **Liveness**: **all requests to enter/exit the critical section eventually succeed** (no deadlock, no starvation)
3. **Optional**: if one request *happened-before* another, then **entry is granted in that order**

We will assume that **processes and channels are reliable**.

Centralized

The **simplest** solution is **emulating a centralized solution**: a server **manages the lock using a token**; resource **access request and release is obtained with respective messages** to the coordinator. Drawbacks: **server is the bottleneck and the single point of failure**.

Scalar clocks

To request access:

1. A process p_i **multicasts a resource request message** m with **timestamp** T_m to all processes
2. Upon **receipt** of m , a process p_j :
 - **If it does not hold** the resource and **is not interested** in holding the resource, **acks**
 - **If it holds** the resource, p_j **puts the request into a local queue ordered according to T_m** (process ids break eventual ties)
 - **If it is also interested** in holding the resource and **has already sent out a request**, it **compares the received timestamp T_m** with the one itself sent. **If T_m is the lowest one**, p_j **acks**, **otherwise it puts the request in the local queue** (like in the point above)

On release, the holding process acknowledges all the requests queued while using the resource. A resource is granted to p_i when its request is acked by all other processes.

Token ring

Processes are **logically arranged in a ring**, regardless of their physical topology. Access is granted by a token that is forwarded along a given direction on the ring:

- A process **not interested** in accessing the resource **forwards** the token
- Resource is **obtained by retaining the token**
- Resource is **released by forwarding the token**

Comparison

Algorithm	Messages per entry	Delay before entry	Problems
Centralized	2	2	Coordinator crash
Lamport	$2(n - 1)$	$2(n - 1)$	Crash of any process
Token ring	From 1 to ∞	0 to $n - 1$	Lost token, process crash

Leader election

Many distributed algorithms require a process to act as a coordinator (or some other special role). The problem is making anyone agree on one. The **minimal assumptions** are that **nodes are distinguishable** and that the **system is closed** (all processes know each other and their IDs but not who is up or has failed). The **criterion for winning** is having the **highest ID**.

Bully election

We have **two additional assumptions**: **reliable links** and the **possibility to decide who has crashed**.

1. When **any process p notices** that the actual **coordinator is no longer responding**, it **initiates an election**
2. p **sends** a **ELECT** including its ID **to all other processes with higher IDs**
3. If **no one responds**, p **wins and sends a COORD message to the processes with lower IDs**
4. If a **process p' with higher id receives an ELECT**, it **responds** (stopping the former candidate) and **starts a new election**
5. If a **process that was previously down comes back up**, it **holds an election**
 - If it happens to be the highest-numbered process, it wins the election and takes over the coordinator's job

Ring-based

Assume a (physical or logical) **ring topology** among nodes. When a **process detects a leader failure**, it **sends** an **ELECT** message containing its ID **to the closest alive neighbour**. Upon **receipt** of the election message a process p :

- If p is **not in the message**, it **adds itself and propagates** to the neighbours
- If p is **in the list**, **changes** the message **to a COORD** and **re-circulates**

Upon **receiving a COORD message**, a **node considers the process with the highest ID as the new leader** (and is also informed about the remaining members of the ring). **Multiple messages may circulate at the same time**, but eventually they **converge to the same content**.

Capturing global state

The global state of a distributed system consists of the **local state of each process together with the message in transit over the links**.

A **distributed snapshot reflects a (consistent, global) state in which the distributed system might have been**. Particular care must be taken **when reconstructing** the global state to **preserve consistency**: if a **message receipt is recorded**, the **message sending must as well**, but the **opposite is not required**. A **distributed snapshot** can be represented by a **cut**.

Formally a **cut of a system** S composed of N processes $p_1, \dots, p_n$ can be **defined as the union of the histories of all its processes up to a certain event**. A cut is **consistent** if **for any event e it includes, it also includes all the events that happened before e** (by the *happens-before* relation).

Chandy-Lamport's algorithm

Assuming FIFO, reliable links and nodes, and a strongly connected graph.

Any process p may initiate a snapshot by recording its initial state, then **sending a token on all outgoing channels** (signals that the snapshot is being run) and then **start recording a local snapshot (i.e. recording messages arriving on every incoming channel)**. Upon **receiving a token**, a process q **starts recording a local snapshot** (like what p did) if it is not already doing so. **Each process considers the snapshot finished when tokens have arrived on all its incoming channels**. Afterwards the **collected data can be sent to a single collector** of the global state.

Theorem The distributed snapshot algorithm selects a consistent cut.

It can be **proven by absurd** (see slides).

The distributed snapshot algorithm **does not require blocking** the computation.

Termination detection

We want to know **when the computation of a distributed system has completed or is deadlocked**. A **computation is completed** if **all processes are idle** and there are **no messages in the system**.

One approach can be taking a **snapshot** a check if all nodes are in a “finished state”.

Another simple solution, proposed by **Tanenbaum**, is similar to the distributed snapshot algorithms but it avoids saving state:

1. We call predecessor of a process p the process q from which it got the first marker. The successors of p are all those processes p sent the marker
2. When a process p finishes its part of the snapshot, it sends a DONE message back to its predecessor only if two conditions are met:
 - All of p 's successors have returned a DONE message
 - p has not received any message between the point it recorded its state and the point it had received the marker along each of its incoming channels
3. In any other case p sends a CONTINUE
4. If the initiator receives all DONE the computation is over; otherwise, another snapshot is necessary

This algorithm appears correct, however the **definition of successor is actually wrong**: a node can be a successor of p even if it is not a predecessor, thus we can have locks.

The correction of Tanenbaum's algorithm is the **Dijkstra-Scholten algorithm**. The algorithm assumes that we are in a case of **diffuse computation**: a process is activated only by a message being sent to it.

1. **Each node keeps track of nodes it sends a message to** (those it may have woken up), its **children**
2. If a **node was already awake** when the message arrived, then it is **already part of the tree**, and should **not be added as a child** of the sender
3. When a node has **no more children and it is idle**, it **tells its parent to remove it as a child**

When only the initiator remains and it has **completed processing**, the system has **terminated**.

Comparison between the two algorithms:

1. Distributed snapshot:
 - Overhead is one message per link and the cost of the state collection
 - If the system has not terminated, we need to run it again
2. Dijkstra-Scholten:
 - Overhead depends on the number of messages in the system
 - Acknowledgments sent when already part of network and when becoming idle
 - A node can be added to network more than once, so this value is not fixed
 - Does not involve never-activated processes

- Termination detected when last ack received

Distributed transactions

Transactions protect a shared resource against simultaneous access by several concurrent processes. They are sequences of operations, defined with appropriate programming primitives like `BEGIN_TRANSACTION` (same for end and abort), `READ`, `WRITE`. They need to respect the ACID properties.

We have **3 types** of transactions:

1. **Flat**: they respect the ACID transactions as we know them
2. **Nested**:
 - **Constructed from sub-transactions**
 - Sub-transactions **can be undone** once committed: durability applies only to top-level transactions
3. **Distributed**:
 - Accounts for data distribution
 - **Essentially flat transactions on distributed data**
 - **Needs distributed locking**

Atomicity

1. Private workspace

1. Copy what the **transaction modifies** into a **separate memory space**, creating **shadow blocks** of the original file
2. If the **transaction is aborted**, this private workspace is **deleted**, **otherwise** they are **copied into the parent's workspace**

We can optimize a bit this approach by replicating the index, not the whole file. Works fine also for the local part of distributed transactions.

2. Writeahead log

1. Files are **modified in place** (commit is fast), but a **log is kept** with the transaction that made the change to which file/block and old/new values
2. **After the log is written** successfully, the **file is actually modified**
3. If **transaction succeeds**, the **commit is written to log**; if it **aborts**, the original **state is restored based on logs**, starting at the end (rollback)

This is a **optimistic approach**, while the **private workspace is a pessimistic approach**.

Controlling concurrency

We have 3 entities interacting:

1. **Transaction manager**: acts as the “**front-end**” (in **distributed** settings we **have one**, acting as a **coordinator**); it transforms high level operations in scheduling requests
2. **Scheduler**: **guarantees consistency and isolation** by determining which transactions can pass an operation to the data manager and when; can be distributed on different hosts
3. **Data manager**: **executes the read/write operations**; it does not know anything about transactions

There are various possible interleavings, only some legal. These interleavings correspond to some linearization of the involved transactions. **Transaction systems must ensure operations are interleaved correctly**, but also **free the programmer from** the burden of programming **mutual exclusion**. This means that it needs to properly schedule to avoid conflicts (read-write and write-write).

We have two approaches: locking vs timestamping.

2-phase locking (2PL) When a process needs to access data it **requests the scheduler to grant a lock**. We need to **enforce two phases**: we have a **growing phase** (transactions acquires locks) and a **shrinking phase** (transaction releases locks) (strict 2PL enforces that locks are released all at once at commit/abort, preventing cascaded aborts). **May lead to deadlocks**. We can implement 2PL in 3 ways:

1. **Centralized 2PL**: the transaction manager contacts a centralized **lock manager**
2. **Primary 2PL**: **multiple lock managers** exist; **each data item has a primary copy** on a host and the lock manager on said host is responsible for granting locks. **The transaction manager is in charge for interacting with the data managers**
3. **Distributed 2PL**: **assumes data may be replicated on multiple hosts**; the **lock manager** on a host is **responsible for granting locks on the local replica** and for **contacting the (local) data manager**; requires **agreement** between replicas

Pessimistic timestamp ordering We assign a **timestamp to each transaction** (e.g. using logical clocks). **Write operations** on a data item x are recorded in **tentative versions**, each with its own write timestamp $ts_w(x_i)$, **until commit** is performed (committed version will have timestamp $ts_w(x)$); each data item also has a **read timestamp** $ts_r(x)$ equal to the last transaction which read x . The scheduler follows these to two rules:

1. When it receives a write on x at time ts , if $ts > ts_r(x) \wedge ts > ts_w(x)$ performs a tentative write, else it aborts
2. When it receives a read on x at time ts we have two cases:
 1. If $ts > ts_r(x)$
 - Let x_{sel} be the latest version of x with the write timestamp lower than ts , if x_{sel} is committed perform the read on it and set $ts_r(x) = \max(ts, ts_r(x))$; else wait for the transaction that wrote on x_{sel} to commit/abort and repeat the check
 - Else abort since the read arrived too late

Aborted transactions will re-apply for a new timestamp and retry. This method is **deadlock-free**.

Optimistic timestamp ordering It is based on the **assumption that conflicts are rare**, therefore we can wait what we want without caring about others and fix conflicts later:

1. **Stamp data items with start time** of transaction
2. **At commit**, if **any items have been changed since start**, transaction is **aborted**, otherwise **committed**

It works best when paired with **private workspaces**. It allows for **maximum parallelism** and **no deadlocks**. However, **under heavy load there may be too many rollbacks**. Not widely used.

Distributed deadlocks

It is the same concept as in conventional systems, but worse to deal with since in a distributed system resources are spread out. **We have 4 ways of dealing with them**:

1. **Ignore** the problem
2. **Detection** and recovery, typically by killing one of the processes
3. **Prevention**: the way our protocol works, it is impossible to have deadlocks
4. **Avoidance**: never used in (distributed) systems, as it implies a priori knowledge about resource usage

In **centralized locking**, each machine maintains a resource graph for its own resources and reports it to a coordinator. We have different options for collecting this information, like:

- Whenever an arc is added or deleted, a message is sent to the coordinator with the update
- Periodically, every process sends a list of arcs added or deleted since the last update
- Coordinator can request information on-demand

None of these methods work well since we could have **false deadlocks** due to bad ordering of messages to the coordinator.

Deadlock detection Here is a completely distributed deadlock detection algorithm with **no coordinator**.

1. Processes are allowed to **request multiple resources simultaneously**
2. **When a process gets blocked**, it sends a **probe message** (e.g. (initiator, sender, receiver)) **to the processes holding resources** it wants to acquire
3. A cycle is detected **if the probe makes it back** to the initiator

To **recover** from a deadlock we have two ways:

1. **Initiator commits suicide** (many processes may be unnecessarily aborted if more than one initiator detects the loop)
2. The **initiator picks the process with the higher identifier and kills it**

- Requires each process to add its identifier to the probe

In practice, **90% of deadlocks in databases involve only two processes.**

Deadlock prevention Make deadlocks impossible by design, for instance by using **global timestamps** (wait-die algorithm):

- When a **process A is about to block for a resource that another process B is using**, allow A to **wait only if A has a lower timestamp** (it is older) than B; **otherwise kill** the process A
 - We could also have the younger wait
- Following a chain of waiting processes, the **timestamps will always increase** (no cycles)

If a process can be **preempted**, an alternative can be devised (**wound-wait** algorithm):

- **Preempting** the young process **aborts its transaction**
- The young process may immediately try to **reacquire the resource, but will wait**
 - In wait-die, the young process may die many times before the old one releases the resource

Distributed agreement in practice

Commit protocols

Atomic commit is a **form of agreement** widely used in database management systems. Commit protocols **ensures atomicity** (duh) **of transactions**. If the **transaction updates data on multiple nodes either all nodes commit or all nodes abort; if any node crashes, all must abort.**

Two-phase commit (2PC)

In 2PC, after the **client has finished sending the commands of the transaction to all the participants it sends a commit message to the coordinator** (which is **also a participant**). The **coordinator then sends** a prepare message to **all the participants**, which **reply with a vote_commit**. After an **agreement** has been reached, the **coordinator sends a global_commit to signal to the participants to write to stable storage**. If **any** of the participants sends a vote-abort or an **agreement cannot be found** by the coordinator, a **global-abort** message will **be sent** to abort the transaction.

When a **participant fails, after a timeout** the coordinator **assumes an abort**. If the **coordinator fails**:

1. Participant **waiting for** prepare can decide to **abort**
2. Participant **waiting for global decision cannot decide on its own**, thus needs to **wait for the coordinator to recover** or can **request the decision to another participant** which **may have received the reply of the coordinator or may be in the initial state** (in which case we assume abort)

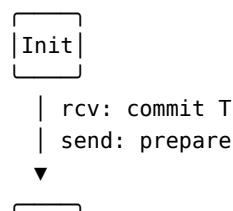
If everybody is in waiting for a global decision we are **stuck** until the coordinator gets back up.

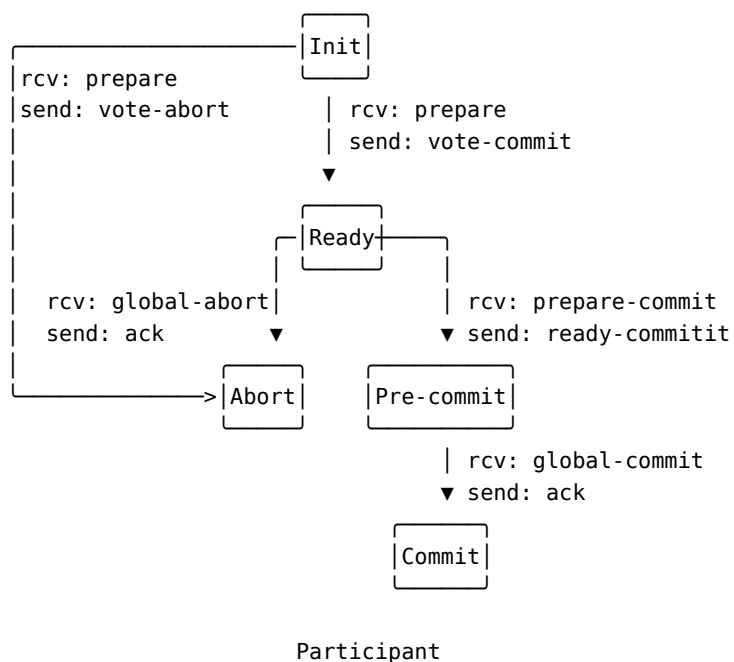
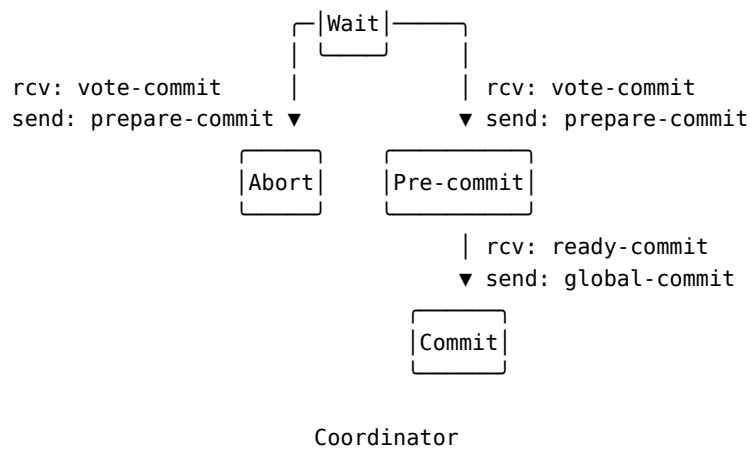
We can then say that 2PC is a **blocking protocol**. 2PC is **safe** (never leads to an incorrect state). In the **case that the coordinator is also participant**, then 2PC is **vulnerable to a single-node failure**.

Three-phase commit (3PC)

It is an attempt to solve the problems of 2PC by **adding another phase to the protocol**. The idea is **splitting the commit/abort phase** in two phases:

1. Communicate the outcome to all nodes
2. Let them communicate only after everyone knows the outcome





If a **participant fails**:

- Coordinator **blocked waiting for vote** (wait state) can **assume abort** decision
- Coordinator **blocked in pre-commit** state can **safely commit** and tell the failed participant to commit when it **recovers**

If the **coordinator fails**:

- Participant **blocked waiting for prepare** (init state) can **decide to abort**
- Participant **blocked waiting for global decision** (ready state) can **contact another participant**. If it receives:
 - At least **one abort**, we **abort**
 - At least **one commit**, we **commit**
 - At least **one init**, we **abort**
 - At least **one in pre-commit** and nodes in **pre-commit** and **ready** form a **majority**, we **commit**
 - **Majority in ready** with **no one in pre-commit**, we **abort**
- **No two participants can be in pre-commit and init**

3PC (quorum-based version presented above) **guarantees safety**, in a **synchronous system** it also **guarantees liveness** (in an asynchronous system the protocol may not terminate). It is **more expensive than 2PC** since it requires three phases of communication.

CAP theorem

Any **distributed system** where nodes **share some (replicated) shared data** can have **at most two** of these three desirable properties:

1. **Consistency** equivalent to have a single up-to-date copy of the data
2. high **Availability** of the data for updates
3. tolerance to network **Partitions**

Replicated state machines

It is a **general purpose consensus algorithm** that **allows a collection of machines (servers) to work as coherent group**. Servers **operate on identical copies of the same state**. They offer a **continuous service**, even if some machines fail, and **clients sees them as a single machine**.

The **state machines responds to external requests and manages internal state**. A **client connects to a leader and sends commands to it**. The **set of operation is kept in a replicated log**, then **consensus is used to agree on the order of operations**.

The **failure model** we will use is:

1. Messages can take arbitrarily long, be duplicate and be lost
2. Processes may fail by stopping and restart, must remember what they were doing (record state) and **NO byzantine failures**

We will **guarantee safety** (all non-failing machines execute the same commands in the same order) and **live-ness/availability** (the system is up if any majority of machines are up and can communicate) (**not always guaranteed in theory, but guaranteed in practice under typical operating conditions**).

Paxos

It was the standard for about 30 years. Problems:

1. Only agreement on a single decision, non on a sequence of requests (solved by multi-Paxos)
2. **Very difficult to understand**
3. Difficult to use in practice (no reference implementation)

Raft

Raft is equivalent to multi-Paxos in terms of assumptions, guarantees and performance. The main design goal was ease of understanding and of usage/adaptability. The main idea is **problem decomposition**. We can identify **3 concerns**:

1. **Log replication**: leader accepts commands from clients, appends to its log and then it replicates its log to other servers
2. **Leader election**: select one server to act as leader, on crashes elect a new leader
3. **Safety**: keep the log consistent by assuring that only nodes with up-to-date logs can become leaders

All **commands go through the leader**, who is responsible for committing and propagating them. Normal operation:

1. Client sends command to a leader, leader appends command to its log
2. **Leader sends AppendEntries to all followers**
3. Once a new entry is committed:
 - **Leader executes command** in its state machine, **returns result** to client
 - **Leader notifies followers** of committed entries in subsequent AppendEntries
 - **Followers execute committed commands** in their state machines

Leader **retries AppendEntries messages until they succeed**. In the **common case performance is optimal**: one successful message to any majority of servers

The leader **periodically sends possibly empty AppendEntries messages** with its unacknowledged log entries. **Followers have a randomized timeout: if they don't hear from the leader until that timeout, they start an election**.

Raft **divides time into terms** of arbitrary length, to help to **identify obsolete information**. Terms are **numbered with consecutive integers**. Each server maintains a **current_term** value and **attaches it** in every communication. Each

term begins with an election, in which one or more candidate try to become leader. There is **at most one leader per term**, if there is a **split vote followers try again when the next timeout expires**. In theory we could go on forever, in practice it doesn't happen.

Nodes can be in **three states: follower, leader and candidate**; all nodes **start as followers**. Followers **wait for a regular heartbeat** from the leader; if **they don't hear a heartbeat for a while**, they **become a candidate**. A **candidate starts an election** by sending a RequestVote. As soon as we become candidate, we **increase the current term** by one and **ask for votes** from the other servers (**we vote for ourselves**). If we **receive the majority of votes**, we **become the new leader**; if we **receive a message from an already existent leader**, we **go back to following**.

The **log is stored on disk** to survive process failures. An **entry is committed to the log by the leader if it is acked by the majority of followers**. Leader always **assumes that its log is correct, normal operation will repair inconsistency**. Raft guarantees the **log-matching property**: if log entries on different servers have the same index and term they store the same command and the logs are identical in all preceding entries. The AppendEntries **message contains the <index, term> of the entry preceding the new one(s)**. The **follower that receives the message rejects the request if it doesn't contain matching entries**; in case of rejection the **leader retries by starting with lower log indexes until success**. Once a log entry is committed, all future leaders must store that entry; this means that **candidates with incomplete logs must not get elected**. **Candidates include the index and term of the last log entry** in RequestVote messages; a **voter can deny the vote if its log is more up to date**.

Use in distributed DBMS

Some modern distributed database management systems integrate replicated state machines and commit protocols.

The typical situation a distributed DBMS has to deal with is:

- The data is partitioned, inter-partition transactions must guarantee atomic commitment
- Each partition is replicated

We work on two layers:

1. Replicated state machine to guarantees that individual partitions do not fail
2. 2PC executes atomic commit across partitions
 - Coordinator (transaction manager) and participants (partitions) are assumed not to fail as they are replicated
 - Channels are assumed to be reliable: it is sufficient that a majority of nodes in a given partition is reachable

Byzantine conditions

State **machine replication can be extended to consider byzantine processes** (Byzantine Fault Tolerant (BFT) replication): it **requires $3f + 1$ participants to tolerate f failures**. We will not see this family of protocols.

Blockchains

Cryptocurrencies can be seen as **replicated state machines where the state is the current balance of each user**. The **state is stored in a replicated ledger (log)**.

The **environment** we are working in is **purely byzantine** (a misbehaving user may try to “double spend” their money and create inconsistent copies of the log). We assume:

1. A **very large number of nodes**
2. The **set of participating nodes is unknown** upfront
3. **No single entity owns the majority of compute resources**

We **can guarantee safety only with high probability**, not certainty.

We will refer to **permissionless systems based on proof of work** (e.g. Bitcoin).

The **blockchain is the public ledger that records transactions**: it **contains all transactions from the beginning of the blockchain with each block of the chain including multiple transactions**. It is a **distributed ledger** (replicated log). **Transactions are signed and published to the bitcoin network**: nodes add them to their copy of the chain and periodically broadcast it.

Adding a block to the chain requires solving a mathematical problem (proof of work): it takes as input the existing chain and a new block; the problem **solution must be difficult to find** (usually only by brute-force) but **easy to verify**.

Proof of work is computed by special nodes (miners) that collect new (pending) transactions into a block. Miners have a **monetary incentive** (prize if they mine successfully a block) to do what they do. **When a miner finds the proof, it broadcasts the new block;** this defines the next block of valid transactions. **The other miners receive it and try to create the next block in the chain,** achieving global agreement on the order of blocks.

What if **two miners find a proof concurrently**? The proof is very complex to compute, thus it is **very unlikely** that two computers will find a solution at the same time. It is also **very difficult for someone to force their desired order of transactions**, since it would require **a lot of compute power**. If **two concurrent versions are created, the one that grows faster** (includes more blocks) **survives**. Still, there may always exist a longer chain we are not aware of as **no one can be 100% sure of a given sequence**.

Peer-to-peer

It is a **paradigm that tries to “take advantage of resources at the edges of the network”** by promoting the sharing of resources and services through direct exchange between peers. All nodes are:

1. **Independent**
2. Both **potential users** and **potential servers**
3. **Dynamic**: they come and go unpredictably
4. With **varying degrees of capabilities**

The **scale** of the system can be **huge** (internet-wide) and **geographically distributed**, thus we have no global view of the system. Nodes are connected usually with TCP channels that form an **overlay network**.

Retrieving resources is a fundamental issue in P2P systems due to their inherent geographical distribution. The problem is **making direct requests towards nodes that can answer them in the most efficient way**. We can distinguish **two ways** of doing it:

1. **Search** for something
2. **Lookup** specific item

The actual data of a lookup can become a burden if the query result are routed through the overlay network. As a **result** we will use is a **reference to the location** from where the data can be retrieved.

Centralized search (Napster)

It was the first p2p file sharing application. The key idea is to share the storage and bandwidth of individual (home) users.

We have 4 commands:

1. join: clients contact **central server**
2. publish: **submit list** of files **to central server**
3. search: **query the server** for someone owning the requested file
4. fetch: **get the file directly from peer**

Query flooding (Gnutella)

There is **no central authority**; this means that we need to find a connection point in the network. Gnutella uses **flooding to search**: each **query is forwarded to all neighbours**, with **propagation limited by a TTL field** in messages.

1. join: client contacts a few other nodes, and they become “neighbours”
 - The **new node connects** to a **well known “anchor” node**
 - Then it **sends a ping** to discover other nodes
 - **pong messages are sent in reply** from hosts offering new connection with the new node (**nodes with many connections will most likely not respond, we do not want a highly connected network** to not bog down search)
 - **Direct connections are then made** to the newly discovered nodes
2. publish: no need

3. search: **ask neighbours**, when found **reply to sender up to the original sender**
4. fetch: **get the file directly from the peer** where it is stored

Gnutella is **fully decentralized and distributes the search cost**. Searching can also be done in many ways. The **main drawback is the flooding algorithm**: if we have a C neighbours and D TTL, each search can cause $C \cdot D$ requests. The **search scope is also huge** (all the nodes in the network) with a **big search time** (proportional to the query TTL). Moreover, the **network is unstable** since nodes often leave.

Hierarchical flooding (Kazaa)

Nodes are **divided into “normal nodes” and “supernodes”**: **queries are flooded between supernodes, normal nodes contact supernodes to do a query**.

1. Join: clients contact a supernode (at some point a client can be promoted to supernode)
2. Publish: send list of files to supernode
3. Search: send query to supernode, supernodes flood queries among themselves
4. Fetch: get the file directly from the peer

The improvement over simple flooding is that **it tries to consider node heterogeneity** (supernodes are the more well-connected so can handle the traffic) **and network locality** (only rumored since kazaa is proprietary). Still we have **no real guarantees on search scope or time**.

Collaborative systems (BitTorrent)

One problem that centralized systems have is availability in case of mass downloads: you need to be able to satisfy the demand. **P2P systems can offload the traffic between different peers that all share the same file**. However, this **requires that there is a high availability of nodes that shares files and not just downloads** (free rides).

BitTorrent allows many people to **download the same file without slowing down everyone else’s download**. It does this by **having downloaders swap portions of a file with one another**, instead of all downloading from a single server. Such **contributions are encouraged because every client trying to upload to other clients gets the fastest downloads**.

1. Join: **contact a centralized “tracker” server** and get a list of peers
2. Publish: **run a tracker server**
3. Search: done **out of band** (use a search engine to find a tracker for the file you want)
4. Fetch: **download chunks** of the file **from peers** (and **upload chunks you have to them**)

BitTorrent **terminology**:

- **Torrent**: metadata file describing the file(s) to be shared
 - Name and size of file(s)
 - Checksum of all blocks (each file is split in fixed-size blocks)
 - Addresses of the tracker and peers
- **Seed**: peer that has the complete file and still offers it for upload
- **Leech**: a peer that has incomplete download
- **Swarm**: all seeders/leeches together
- **Tracker**: server that keeps track of seeds and peers in the swarm and gathers statistics

Peers **download missing fragments** from each other and **upload to those who don’t have it**. The fragments are **not downloaded in sequential order** and need to be assembled by the receiving machine. When a client needs to choose which segment to request first, it usually **adopts a “rarest-first” approach**, by identifying the fragment held by the fewest of its peers. This tends to keep the number of sources for each segment as high as possible, **spreading load**. Clients **start uploading what they already have** (small fragments) **before the whole download is finished**. Everyone can eventually **get the complete file as long as there is “one distributed copy”** of the file in the network, **even if there are no seeds**.

Choking is a **refusal to upload**; choking evaluation is performed **every 10 seconds**. Each peer **un-chokes a fixed number of peers**. The **decision** on which peers to un/choke is **based solely on download rate**, which is evaluated on a rolling, 20-second average. This is a **“tit-for-tat”** sharing strategy (I give you something and you give something back).

A BitTorrent peer **has also a (single) “optimistic un-choke”**, which is **uploaded regardless of the current download rate from it**. This **allows discovery of new connections** and also prevents the initial stall. This approximates Pareto

efficiency: if two peers get poor download rates for the uploads they are providing, they can start uploading to each other and get better download rates than before.

Secure storage (Freenet)

Freenet is a P2P application designed to **allow anybody to publish and read information with reasonable anonymity**. Files are **encrypted, signed and referred to by a cryptographic hash**.

1. Join: **clients contact a few other nodes they know about** and get a unique node id
2. Publish: **route file contents** toward the **node which stores other files whose id is closest to the file id**
3. Search: **route a query** for file id using a **steepest-ascent-hill-climbing** search with backtracking
4. Fetch: when query reaches a node containing the queried file id, it **returns the file to the sender**

Nodes have **approximate knowledge of what their neighbors store**. Requests are **forwarded to node's "best guess" neighbor** unless it has the information locally. If the information is found within the request's "hops to live", it is passed back through this chain of nodes to the original requestor. The **intermediate nodes store the information in their LRU cache as it passes through**.

Distributed hash tables (Chord)

It is a distributed data structure that has 2 operations: $\text{put}(\text{id}, \text{item})$ and $\text{item} = \text{get}(\text{id})$. The **nodes are organized in an overlay network**. We can also define the **usual 4 operations**:

1. Join: the **clients contact a "bootstrap" node** and integrate into the distributed data structure by getting a node id
2. Publish: **route** publication for file id **toward a close node id** along the data structure
3. Search: **route** a query for file id **towards a close node id**
4. Fetch:
 - If the **publication contains the actual file**, fetch it
 - If the **publication contains a reference**, fetch it from the reference

The implementation of a DHT we are going to see is Chord.

Nodes and keys are organized in a **logical ring**. Each **node** is assigned a **unique m-bit identifier** (usually hash of the IP) and **keeps track of its successor and predecessor**; every **item** is assigned a **unique m-bit key** (usually hash of the item). The item with **key k** is **managed by the node with smallest id greater than k** .

Each node maintains a **finger table with m entries**. Entry i in the finger table of node n is the **first node whose id is higher or equal than $n + 2^i$** . Upon receiving a query for an item with key k , a node:

1. **Checks whether it stores the item locally**
2. If not, **forwards the query to the node in its successor table that is responsible for the interval of keys including k**

When a **new node n joins**, the following actions must be performed:

1. **Initialize the predecessor and fingers** of node n
 - We assume that n already knows another node n' already in the system: the i -th finger is the successor of node n plus 2^i
2. **Update the fingers and predecessors** of existing nodes to reflect the addition of n
 - We observe that **node n will become the i -th finger of node p iff: p precedes n by at least 2^{i-1} and the i -th finger of p succeeds n** . The **first node p that meets these two conditions** is the **immediate predecessor** of node $n - 2^{i-1}$.
 - We need to do this for each finger i , meaning we have $\log^2(N)$ for joining (fingers are $\log(N)$)

The **correctness** of Chord **relies on the correctness of successors pointers**. To **stabilize routing** we need **periodic procedures** to update successor and fingers.

	Complexity
Routing table size	$\log(N)$ fingers with $N = 2^m$ nodes
Routing time	$\mathcal{O}(\log(N))$
Joining time	$\mathcal{O}(\log^2(N))$

The main **pros** of Chord are the **logarithmic per node state and search scope**. However it is unused, more **fragile** than unstructured networks and **dismisses the physical topology** of the underlying network. Moreover we have **only key search**.

Replication

Replication is used for **achieving fault tolerance (redundancy)** or to **increase availability** (leveraging **data locality**, or **load balancing**) or **improving performance** (again, by load balancing, replication to **improve throughput**, or leveraging data locality).

Latency does not improve as other performance metrics do; there comes a point when latency becomes the bottleneck.

The **main problem** of replication is **consistency**: when we change a replica, we need to change all the others; when we have **multiple simultaneous updates** we can have **read/write conflicts** and need to deal with them. **Replication may decrease the overall performance** since the cost to ensure that data is consistent across replicas can be high. Depending on the type of application (data/client centric) we have different consistency requirements.

Models

Ideally, a read should show the result of the “last” write; but what does “last” mean? Without a global clock we cannot determine it.

A **consistency model is a contract between the processes and the data store**. Stricter guarantees simplify the development but incur higher costs; weaker guarantees reduce the cost but make development difficult. It is a **trade-off between guarantees, performance and ease of use**.

Types of models:

- **Guarantees on content**: maximum “difference” on the versions stored at different replicas
- **Guarantees on staleness**: maximum time between a change and its propagation to all replicas
- **Guarantees on the order of updates**: constrain the possible behaviors in the case of conflicts

Consistency protocols implement consistency models. In this context, we say that a **protocol is highly available if it does not require synchronous/blocking communication**. By the CAP theorem, **if we have availability we can have tolerance or consistency**, thus **highly-available protocols are usually weakly-consistent**. Inversely, **weakly available protocols are usually strongly-consistent**.

Single leader protocols

One of the replicas is designated as the **leader**. When **clients want to write** to the datastore, they **must send the request to the leader**, which **first writes the new data to its local storage**. The **other replicas** are **followers**. Whenever the **leader** writes new data to its local storage, it also **sends the data to all its followers**.

When a **client wants to read** from the database in **some systems it queries directly the leader**, while in others it can **query any replica**.

A single leader protocol can be:

1. **Synchronous**: the **write operation completes after** the leader has received a **reply from all the followers**
2. **Asynchronous**: the **write operation completes** when the **new value is stored on the leader**; **followers are updated asynchronously**
3. **Semi-synchronous**: The write operation **completes** when the leader has **received a reply from at least k replicas** (k is configuration parameter)

Synchronous or semi-synchronous protocols are safer since even if $k - 1$ replicas fail, we still have a copy of the data; **followers can recover by “catching up”** to other replicas. If the **leader fails** we can **elect a new one** (fail-over). This can bring many tricky situations depending on the assumptions.

Single leader protocols with synchronous or semi-synchronous replication **widely adopted in distributed databases**. They are used inside a single organization/data center since it **benefits greatly from low latency and can spread frequent reads across replicas**.

No write-write conflicts possible since the leader receives all write operations and determines their order. Read-write conflicts still possible depending on the specific implementation.

Multi-leader protocols

Writes are carried out at different replicas concurrently. No single leader means that there is no single entity that decides the order of writes: this means that it is possible to have write-write conflicts.

Multi-leader protocols are often adopted in geo-replicated settings and in general are more difficult to handle. In practice the conflicts are rare and easy to solve in several application scenarios.

Multi-leader protocols natively supported in some database in addition to single leader protocols. Sometimes they are implemented on top of DBMSs in external tools.

Leader-less protocols

In leaderless replication, the client contacts multiple replicas to perform the writes/reads (in some implementations, a coordinator forwards operations to replicas on behalf of the client). Leaderless replication uses quorum-based protocols to avoid conflicts.

It is used in some modern key-value stores.

Data-centric models

1. **Sequential consistency**: “The result is the same as if the operations by all processes were executed in some sequential order, and the operations by each process appear in this sequence in the order specified by its program”

It is a very strong (thus not very available) model that is very expensive in terms of performance.

Since all replicas need to agree on a given order, we can use synchronous single-leader protocols. This works if links are FIFO and clients are “sticky” (they always read from the same replica).

Another implementation uses leaderless system and a quorum-based algorithm: clients contact multiple replicas to perform a read or a write operation. An update occurs only if a quorum of the servers agrees on the version number to be assigned; reading requires a quorum to ensure the latest version is being read. The quorums (NR , NW is the quorum for respectively reading or writing, N is the number of nodes) usually follow:

- $NR + NW > N$ to avoid read-write conflicts
- $NW > \frac{N}{2}$ to avoid write-write conflicts

Replicas are updated in two ways:

- **Read-repair**: when a client makes a read from several nodes in parallel, it can detect any stale responses; it sends the new value to the replicas that are not up-to-date
- **Anti-entropy**: In addition to read-repair, nodes periodically exchange data in background to remain up-to-date

Sequential consistency, in both implementations, is not highly available and has high latency. In case of network partitions, clients are blocked.

2. **Linearizability**: “Each operation should appear to take effect instantaneously at some moment between its start and its completion”

Also known as strong/external/atomic consistency, it is the strongest possible consistency guarantee. Linearizability gives the illusion of having a single copy of the data store.

Linearizability includes a notion of time: operations behave as if they took place at a single point of (wall clock) time, unlike sequential consistency. It may require time to propagate an operation to all replicas, so the operation may not be instantaneously visible.

Linearizability is a composable property (i.e. if the operations on individual variables are linearizable the global schedule is also linearizable).

Single leader replication may be used to implement linearizability: the **leader orders writes according to their timestamp, replicas are updated synchronously and atomically**. This is **very difficult** to guarantee in the **presence of failures** (it is an agreement/consensus problem to determine if and how the network is partitioned and who is the leader).

3. **Causal consistency**: “Writes that are **potentially causally related** must be **seen by all processes in the same order**. **Concurrent writes** may be seen in **any order** at different machines”

It is a **highly available model**. It **weakens sequential consistency** based on Lamport’s notion of *happened-before*, only difference is that Lamport’s model deals with message passing while we deal with read/writes.

The causal order is defined as follows:

- A **write** operation W by a process P is **causally ordered after every previous operation O by the same process**, even if W and O are performed on different variables
- A **read** operation by a process P on a variable x is **causally ordered after a previous write by P on the same variable**
- Causal order is **transitive**

It is not a total order since operations that are not causally ordered are said to be concurrent. **Sequential consistency implies causal consistency**.

Multi-leader implementations are possible: we **timestamp writes with vector clocks**. The implementation with multi-leader **works only if clients are sticky**.

4. **FIFO consistency**: “Writes done by a **single process are seen by all others in the order in which they were issued**; writes from different processes may be seen in any order at different machines”

Also called **PRAM (pipelined RAM) consistency**, **causality across processes is dropped**: if writes are put onto a pipeline for completion, a process can fill the pipeline with writes, not waiting for early ones to complete.

It is **very easy to implement** (even in a multi-leader situation): just use a **scalar clock**.

Causal consistency implies FIFO consistency.

Eventual consistency

It is not a data-centric model, but is widely used when:

1. There are **no simultaneous writes**, or they can be **easily resolved**
2. **Mostly reads**

Examples: Web caches, DNS, social media (geo- distributed data stores). In these systems, eventual consistency is often sufficient: **updates are guaranteed to eventually propagate to all replicas**.

It is very popular today because:

1. It is very easy to implement
2. In practice, there are very few conflicts (e.g. today’s networks are fast enough for propagation)
3. Dedicated data-types, called **conflict-free replicated data types (CRDTs)**, that **guarantee convergence even if updates are received in different orders**:
 - Integer counter where replicas store the **set of increase/decrease ops**
 - **Append only** data structure
 - **Last-write-wins approaches**: clients attach a unique random identifier to each append; in case of conflicts, the write with the larger identifier is considered the last one

Client-centric models

What happens if a **client dynamically changes the replica it connects to**? This problem is addressed by **client-centric consistency models** that **provide guarantees about accesses to the data store from the perspective of a single client**.

1. **Monotonic reads**: “If a **process reads the value of a data item x**, any **successive read operation on x by that process will** always return **that same value or a more recent value**”

2. **Monotonic writes:** “A **write operation** by a process on a data item x is **completed before any successive write operation** on x by the same process”

Similar to FIFO consistency, although this time for a single process. A weaker notion where ordering does not matter is possible if writes are commutative.

3. **Read your writes:** “The **effect of a write operation** by a process on a data item x will **always be seen by a successive read operation** on x by the same process”
4. **Write follows reads:** “A **write operation** by a process on a data item x **following a previous read operation** on x by the same process is **guaranteed to take place on the same or more recent value** of x that was read”

For implementing client centric models, we **assign to each operation a unique identifier**. **Two sets** are defined for each client:

1. **Read-set:** the **write identifiers relevant for the read operations** performed by the client
2. **Write-set:** the **identifiers of the writes performed by the client**

These identifiers can be **encoded as vector clocks**.

Design strategies

We have seen some protocols to keep replicas consistent with respect to some consistency model. There are further issues in designing a replicated datastore, including:

- **How to place replicas?**
 - **Permanent** replicas, statically configured (see DNS, CDNs etc)
 - **Server-initiated** replicas: they are created dynamically to cope with access load or to move data closer the clients; often requires topological knowledge
 - **Client-initiated** replicas: rely on client case, that can be shared among clients for enhanced performance
- **What to propagate?**
 - **Perform the update and propagate only a notification:** if used in conjunction with invalidation protocols avoids unnecessarily propagating subsequent writes; it has small communication overhead and works best if $\#reads \ll \#writes$
 - **Transfer the modified data to all copies:** works best if $\#reads \gg \#writes$
 - **Propagate information to enable the update operation to occur at the other copies (active replication):** very small communication overhead, but may require unnecessary processing power if the update operation is complex; we may need to deal with possible side-effects
- **How to propagate updates between them?**
 - **Push-based approach:** the update is propagated to all replicas, regardless of their needs; typically used to preserve a high degree of consistency
 - **Pull-based approach:** an update is fetched on demand when needed; core convenient if $\#reads \ll \#writes$ and typically used to manage client caches
 - **Lease:** used to switch between pull and push based approaches

Case studies

Distributed databases

Broadly speaking, modern systems can be **classified based on the decision they take with respect to the CAP theorem**. As network partitions may always occur, systems may choose to offer either:

1. **Strong guarantees (C) through blocking communication**
 - Higher latency, potentially not available in the presence of failures
 - **NewSQL** databases are newer SQL database that can work in distributed settings:
 1. Isolation: implemented through locking and timestamping
 2. Atomicity: blocking commit protocols
 3. Replication: sequential consistency or linearizability
2. **High availability (A) through non-blocking communication**
 - Lower latency, lower guarantees
 - They are **NoSQL** (Key-value stores, wide-columns, document stores) databases where:

1. Isolation is not needed since we do not have multi-item transactions
2. Atomicity is not needed since we do not have multi-item transactions
3. Replication is asynchronous or multi-leader

Now we are going to see some examples of DBs with strong guarantees and how they manage the cost of the protocols they use.

Spanner

It is designed for very large data bases and uses standard techniques:

1. **Single leader replication with Paxos** for fault-tolerant agreement on followers and leader
2. **2PC** for atomic commit
3. **Timestamp** protocols for concurrency control

The way to cut down on costs is TrueTime: it uses **very precise clocks** (atomic/GPS) that offer an API that return an **uncertainty range** with the “real time” surely inside it.

Read-write transactions use TrueTime to decide when to commit:

1. The **transaction coordinator asks a transaction timestamp** to TrueTime
2. It **waits to release all locks and commit only when the uncertainty range is certainly passed**, thus the commit timestamp is certainly passed for every node
3. Since **transactions are ordered based on time**, it implements **linearizability**

Read-only transactions also acquire a timestamp through TrueTime, but they do **not need to lock**, but simply read the latest value at that time.

Calvin

Designed for the same settings as Spanner, **adopts a sequencing layer** to order all incoming requests (read and write). This layer is **replicated for durability using a replicated logs with Paxos**.

It guarantees **linearizability** (provided by the sequencing layer). The advantage it provides are:

1. **Agreement (order of execution) achieved before acquiring locks**, meaning lower lock contention
2. **No need for 2PC**: as transactions are deterministic, they either succeed or fail in all replicas

VoltDB

Developers specify how to partition database tables and transactions.

It **guarantees sequential partitions**, however **single-partition transactions may execute sequentially on that partition without coordinating** with other partitions, making them very fast. For other transactions, **standard protocols are used**.

Big data platforms

From a technical perspective we have the **following problems**:

1. **Collecting** all the data
 - The more the better
 - Keeping everything is cheaper than deciding what to keep and what to discard
2. **Decide** independently **what to do with data**

An need to face the **following challenges**:

1. **Volume**: amount of data
2. **Velocity**: amount of data entering the system
 - In many domains, information is relevant when it's fresh and loses value as it becomes old, thus we need to process data and extract valuable knowledge as soon as possible!
3. **Variety**: data can have many different formats, versions and sources
4. **Veracity**: data is not always correct

We need to scale to large data volumes and support quickly changing data. The **required functions** are:

1. **Automatic parallelization and distribution**
2. **Fault-tolerance**
3. **Status and monitoring tools**
4. **A clean abstraction** for programmers

MapReduce

We make some **assumptions about the computing infrastructure**:

1. We are working with a **cluster of “normal” computers**
2. We have hundreds to **thousands of nodes**
3. **No dedicated hardware**
4. **Failures are common**

And on the scale:

1. **Terabytes or more of data**, too much to fit on a single disk, or too expensive to read from a single disk
2. Data **doesn’t fit in memory**

MapReduce is a **programming model** introduced by Google. It enables application programs to be written in terms of **high-level operations on immutable data**. The **runtime system controls scheduling, load balancing, communication, fault tolerance, etc....**

Computation is split in **two phases**: map and reduce:

1. **Map processes individual elements** and for each of them **outputs one or more** (key, value) **pairs**
2. **Reduce processes all the values with the same key and outputs a value.**

The **developers need only to specify the behaviour of these two functions.**

Scheduling

We have **one master and many workers**. **Input** data is **split** into M map tasks. The **reduce phase** is **partitioned** into R reduce tasks. **Tasks are assigned to workers dynamically.**

The **master assigns each map task to a free worker**, considering **data locality** to worker. The mapping worker **reads the task input** (often from a local disk) and produces R **local files containing the intermediate key-value pairs.**

The **master assigns each reduce task to a free worker**. The reducing worker will **read intermediate key-value pairs** from map workers and then **sorts and applies the user’s reduce operation** to produce the output.

The filesystem needs to be able to handle the workload and distribute files evenly across nodes. In **GFS** (Google’s FileSystem), **data files are divided into 64MB blocks** and **3 copies of each are stored on different machines.**

Fault-tolerance

The **master detects worker failures via a heartbeat**. On **failure**, it **reallocates the completed and in-progress map tasks to another worker** (since partial results are stored locally). For **reduce tasks, only in-progress ones** should be re-executed, since the output is stored in the global filesystem.

State is check-pointed periodically in GFS by the master. **On master failure we wait for the master to recover and we continue.**

Stragglers

“Stragglers” are **tasks that take a long time to execute** due to various reasons like slow hardware, poor partitioning bugs and other. To improve performance, we **duplicate the struggling tasks to free workers** and keep the version that finishes first.

Conclusions

- **Strengths:**
 - The **developers only write simple functions**, while the system handles all other details
 - Very **general**
 - **Good for large scale data analysis**
- **Limitations:**
 - **High overhead**
 - **Lower raw performance** than HPC
 - Very **fixed paradigm**: each step must complete before the next one can start

Beyond MapReduce

In the last decade, many systems **extended and improved the MapReduce abstraction** in many ways:

1. From two processing steps to **arbitrary acyclic graphs of transformations (dataflow model)**
 - Processing is split into tasks. Transformations that do not reshuffle elements can be grouped in the same task
2. From **batch processing to stream processing**
 - Not only process large datasets with high throughput but also low latency
3. From **disk to main-memory or hybrid** approaches

We consider two systems that differ in how/when they allocate tasks onto nodes:

1. Apache Spark: batch processing + scheduling of tasks
2. Apache Flink: streaming/continuous processing + pipelining of tasks

Apache Spark

Similar to MapReduce, but it is **composed of an arbitrary number of stages** instead of only 2. **Intermediate results can be cached in main memory** if they are reused multiple times. **Scheduling of tasks (stages) ensures that the computation takes place close to the data.** There is **support for streaming data through the micro-batch approach**: input data stream is **split into small batches of data** that are **processed independently with some state that can persist across batches.**

Apache Flink

A job is not split into stages, but instead **all operators are instantiated as soon as the job is submitted.** Operators **communicate with each other through TCP** and **start processing as soon as it has some data available from the previous ones (pipeline architecture).** This architecture is ideal for stream processing since **data flows into the system without waiting for scheduling, allowing lower latency.** The same approach **can be used for batch processing: stream the entire batch through the operators.**

Comparisons

1. **Latency: the pipeline approach of Flink provides lower latency**
 - New data elements are ingested into the network of processing operators as they become available
 - No need to accumulate (micro-)batches or schedule tasks
2. **Throughput: a scheduling approach offers more opportunities to optimize throughput**
 - Moving larger data blocks can be more efficient due to lower network protocol overheads and opportunities for data compression
 - Scheduling decisions can consider data distribution
3. **Load-balancing: a scheduling approach simplifies load balancing since dynamic scheduling decisions can consider data distribution**
 - This is not possible in the case of a pipelined approach as the allocation of tasks to operators is decided statically when the job is deployed
4. **Elasticity: elasticity indicates the possibility of a system to dynamically adapt resource usage to the load of the system**
 - **Scheduling approaches are better for elasticity** since scheduling **decisions take place dynamically at run-time** and thus are able to adapt to changes in the set of physical resources

- Elasticity is simply **not possible in pipelined approaches**; the only way is **snapshotting the system and restarting it on a different set of physical nodes**

5. **Fault-tolerance:**

- In case of a **scheduling-based** approach we simply **reschedule the tasks** of a failed node
 - Spark relied on “lineage”: no replication of intermediate results, if a data element is lost recompute it, if the data it depends on is lost recompute that and so on
- In **pipelined processing** we need to **periodically checkpoint to (distributed) file system** and **replay** from the last checkpoint **in case of failure**